

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский
Нижегородский государственный университет
им. Н.И. Лобачевского (ННГУ)»**

Институт информационных технологий, математики и механики

**Отчет по лабораторной работе
по курсу**

“Параллельные численные методы”
“Метод сопряженных градиентов”

Выполнил:

студент группы 3825М1ПМкн1
Сучков В.Н.

Проверил:

д.т.н., проф. кафедры МОСТ
Баркалов К.А.

Нижний Новгород
2025

Содержание

1	Введение	2
2	Постановка задачи	2
2.1	Формат входных и выходных данных	2
2.2	Критерии успешности и ограничения	3
3	Метод решения задачи	4
3.1	Математический алгоритм	4
3.2	Особенности умножения в симметричном CRS	4
4	Описание реализации алгоритма	5
4.1	Используемые структуры данных	5
4.2	Описание базовой логики программы	5
5	Различные способы распараллеливания и их сравнение	6
5.1	Варианты декомпозиции данных	6
6	Реализация параллельного алгоритма с использованием общей памяти	7
6.1	Организация параллельных секций	7
6.2	Синхронизация и последовательные участки	7
6.3	Параллельная обработка разреженной структуры	7
7	Описание тестовых задач	8
7.1	Тесты на проверку корректности	8
7.2	Тесты производительности и масштабируемости	8
8	Результаты экспериментов	9
8.1	Проверка корректности	9
8.2	Тесты производительности и масштабируемости	10
9	Заключение	10
10	Список литературы	11
A	Приложение. Код	12

1 Введение

Целью данной работы является реализация, тестирование и исследование производительности параллельного алгоритма метода сопряженных градиентов для решения СЛАУ вида $Ax = b$, где $A = A^T > 0$.

В современных вычислительных задачах (например, при решении дифференциальных уравнений в частных производных сеточными методами) матрицы систем достигают огромных размеров, однако подавляющее большинство их элементов равны нулю. Хранение таких матриц в плотном формате нецелесообразно из-за колоссальных затрат памяти и выполнения множества избыточных операций умножения на ноль. В связи с этим в работе применяется формат сжатого хранения CRS. Для дополнительной оптимизации, учитывая симметрию матрицы, в памяти хранятся только элементы, расположенные не ниже главной диагонали.

Метод сопряженных градиентов относится к классу итерационных методов и является одним из наиболее эффективных способов решения СЛАУ с симметричными положительно определенными матрицами. Основной вычислительной нагрузкой алгоритма является операция умножения разреженной матрицы на плотный вектор. Для ускорения расчетов и распараллеливания вычислений в системах с общей памятью применяется фреймворк OpenMP.

Особую сложность при распараллеливании представляет структура симметричного формата CRS: при умножении матрицы на вектор возникает необходимость одновременной записи результатов вычислений симметричных внедиагональных элементов разными потоками в одни и те же ячейки результирующего вектора. Устранение возникающего состояния гонки данных и эффективное распределение нагрузки между вычислительными ядрами является ключевой задачей данной реализации.

2 Постановка задачи

Требуется реализовать на языке C++ с использованием технологии OpenMP метод сопряженных градиентов для решения СЛАУ вида:

$$Ax = b \tag{1}$$

где $A = A^T > 0$ — разреженная матрица размерности $n \times n$, а x и b — плотные векторы размера $n \times 1$.

Программа должна реализовывать функцию со следующим заголовком:

```
void SLE_Solver_CRS(CRSMatrix & A, double * b, double eps,
                   int max_iter, double * x, int & count);
```

2.1 Формат входных и выходных данных

- Входные параметры:

- `A` — ссылка на структуру `CRSMatrix`, содержащую матрицу в симметричном формате CRS (хранятся только элементы не ниже главной диагонали, т.е. $j \geq i$).
- `b` — указатель на массив правой части СЛАУ размера n .
- `eps` — критерий остановки итерационного процесса по относительной норме изменения решения.
- `max_iter` — максимальное число итераций.

- **Выходные параметры:**

- `x` — указатель на массив для записи результирующего вектора решения.
- `count` — количество фактически выполненных алгоритмом итераций.

Матрица A представляется специализированной структурой:

```
struct CRSMatrix {
    int n;           // Число строк в матрице
    int m;           // Число столбцов в матрице
    int nz;          // Число ненулевых элементов в верхнем треугольнике
    vector<double> val; // Массив значений элементов по ( строкам)
    vector<int> colIndex; // Массив номеров столбцов
    vector<int> rowPtr; // Массив индексов начала строк
};
```

2.2 Критерии успешности и ограничения

- **Ограничения на размер задачи:** размерность матрицы $n \leq 100000$, общее число ненулевых элементов $nz \leq 10^7$. Лимит по времени: 100 с. Лимит по памяти: 256 МБ.
- **Критерий остановки внутри цикла:**

$$\frac{\|x_k - x_{k+1}\|_2}{\|b\|_2} < \text{eps} \quad (2)$$

или превышение лимита итераций `max_iter`.

- **Критерий корректности ответа:** Относительная норма невязки на выходе должна удовлетворять условию:

$$\frac{\|Ax - b\|_2}{\|A\|_2} < 10^{-8} \quad (3)$$

- **Требование к масштабируемости:** Эффективность параллельной версии должна составлять не менее 50% на больших тестах.

3 Метод решения задачи

Для решения СЛАУ применяется итерационный метод сопряженных градиентов. Метод строит последовательность приближений x_k и сходится за конечное число шагов.

3.1 Математический алгоритм

Пусть задано начальное приближение $x_0 = 0$. Тогда инициализация алгоритма имеет вид:

$$r_0 = b - Ax_0 = b$$

$$p_0 = r_0$$

На каждой итерации $k = 0, 1, \dots, \text{max_iter} - 1$ выполняются следующие вычисления:

1. Вычисление матрично-векторного произведения: $A_p = A \cdot p_k$.

2. Вычисление шага вдоль направления:

$$\alpha_k = \frac{(r_k, r_k)}{(A_p, p_k)} \quad (4)$$

3. Обновление вектора решения: $x_{k+1} = x_k + \alpha_k p_k$.

4. Проверка критерия остановки по норме приращения решения.

5. Обновление невязки: $r_{k+1} = r_k - \alpha_k A_p$.

6. Вычисление коэффициента сопряженности:

$$\beta_k = \frac{(r_{k+1}, r_{k+1})}{(r_k, r_k)} \quad (5)$$

7. Построение нового направления: $p_{k+1} = r_{k+1} + \beta_k p_k$.

3.2 Особенности умножения в симметричном CRS

Наиболее трудоемкой операцией метода является произведение разреженной матрицы на вектор $A \cdot p_k$. Так как в структуре `CRSMatrix` хранятся только элементы верхнего треугольника ($j \geq i$), стандартный обход строк не позволит вычислить полное произведение.

Для корректного умножения каждый элемент $v = A_{ij}$ ($i \neq j$) обрабатывается дважды:

- Прямой вклад в строку i : $\text{sum} += v \cdot p_j$
- Симметричный вклад в строку j : $A_p[j] += v \cdot p_i$

Такой подход минимизирует расходы памяти, но порождает серьезную проблему для параллельной реализации, так как разные потоки могут одновременно пытаться обновить одну и ту же ячейку массива $A_p[j]$. Описание борьбы с этой проблемой приведено в следующих разделах.

4 Описание реализации алгоритма

4.1 Используемые структуры данных

Для эффективной работы с разреженными матрицами больших размерностей в программе используется симметричный формат Compressed Row Storage. В отличие от классического плотного представления, данный формат позволяет хранить исключительно ненулевые элементы структуры, расположенные на главной диагонали и выше нее (в силу симметрии матрицы A).

Память под матрицу выделяется динамически в рамках структуры `CRSMatrix`:

- `val` — одномерный массив вещественных чисел типа `double`, содержащий значения ненулевых элементов верхнего треугольника матрицы, записанные последовательно по строкам.
- `colIndex` — одномерный массив целых чисел типа `int`, хранящий номера столбцов для каждого соответствующего элемента из массива `val`.
- `rowPtr` — одномерный массив целых чисел типа `int` размера $n + 1$, где элемент с индексом i указывает на позицию начала i -й строки в массивах `val` и `colIndex`. Последний элемент `rowPtr[n]` равен общему числу ненулевых элементов верхнего треугольника (`nz`).

Вспомогательные векторы r (текущая невязка), p (направление), A_p (результат умножения матрицы на вектор направления), а также вектор искомого решения x реализуются стандартными динамическими массивами `std::vector<double>` и располагаются в памяти непрерывно.

4.2 Описание базовой логики программы

Реализация алгоритма сосредоточена в функции `SLE_Solver_CRS` и разбита на три ключевых этапа:

1. **Инициализация и выделение ресурсов:** Начальное приближение x_0 заполняется нулями. Вектор остатка r и начальное направление спуска p инициализируются значениями вектора правой части b . Вычисляется квадрат нормы вектора правой части $\text{norm_b_sq} = (b, b)$ и его скалярный корень для последующих проверок сходимости. Если норма равна нулю, программа мгновенно завершает работу, возвращая нулевой вектор.
2. **Выделение локальных буферов потоков:** Для предотвращения конфликтов параллельного доступа создается двумерный массив `local_Ap` размера $\text{max_threads} \times n$. Каждому выполняющемуся потоку сопоставляется собственный изолированный вектор для накопления частичных результатов матрично-векторного умножения.

3. **Основной итерационный цикл:** Цикл выполняется до достижения `max_iter` или досрочного выполнения критерия остановки. Каждая итерация включает в себя:

- Обнуление локальных буферов `local_Ap` для текущего шага.
- Вычисление произведения $A \cdot p$ с учетом симметрии матрицы.
- Редукцию (суммирование) локальных буферов всех потоков в глобальный вектор A_p .
- Вычисление скалярного произведения (A_p, p_k) и корректировку шага α_k .
- Модификацию решения x и вычисление нормы его приращения Δx .
- Обновление вектора невязки r , расчет коэффициента β_k и пересчет направления p .

5 Различные способы распараллеливания и их сравнение

5.1 Варианты декомпозиции данных

При распараллеливании операции умножения разреженной симметричной матрицы на плотный вектор $(A \cdot p)$ по строкам возникает проблема гонки данных.

Рассмотрим два альтернативных подхода к организации вычислений:

1. **Прямое распараллеливание с атомарными операциями:** Потоки распределяют итерации внешнего цикла по строкам i . Внутри цикла элемент A_{ij} умножается на p_j и добавляется в локальную сумму строки i . Симметричный вклад

$A_{ij} \cdot p_i$ должен быть добавлен в ячейку $A_p[j]$. Поскольку разные строки (и, соответственно, разные потоки) могут иметь элементы, относящиеся к одному и тому же столбцу j , запись в $A_p[j]$ требует синхронизации (например, через директиву `#pragma omp atomic`). Эксперименты показывают, что высокая плотность атомарных операций катастрофически снижает производительность из-за блокировок шины данных.

2. **Декомпозиция с использованием частных буферов (выбранный подход):** Каждому потоку выделяется индивидуальный вектор `local_Ap[tid]` размера n . Потоки полностью изолированы друг от друга в процессе обхода матрицы. Вклад от внедиагонального элемента записывается потоком в свой собственный буфер: `local_Ap[tid][j] += v * p[i]`. Гонка данных полностью исключается на этапе расчета, а накладные расходы сводятся к финальной параллельной сборке (редукции) буферов, выполняемой за $O(n_threads \cdot n)$ операций.

6 Реализация параллельного алгоритма с использованием общей памяти

6.1 Организация параллельных секций

Для минимизации накладных расходов на повторное создание и уничтожение потоков, основной вычислительный цикл по итерациям метода сопряженных градиентов помещен внутрь единой параллельной области `#pragma omp parallel`.

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int n_threads = omp_get_num_threads();
    // Дальнейший итерационный цикл по шагам метода
}
```

6.2 Синхронизация и последовательные участки

Внутри основной параллельной области явные барьеры синхронизации отсутствуют, поскольку логика алгоритма опирается на неявные барьеры, встроенные в директивы распределения циклов `#pragma omp for`.

Потоки синхронизируются в следующих критических точках:

- **После обнуления локальных буферов:** Перед началом умножения матрицы на вектор гарантируется, что все потоки завершили очистку своих строк в `local_Ap`.
- **После этапа умножения разреженной матрицы на вектор:** Перед началом финального суммирования все частичные вклады потоков должны быть полностью вычислены.
- **После сборки глобального вектора A_p :** Неявный барьер в конце цикла сборки гарантирует, что к моменту перехода к вычислению скалярного произведения (A_p, p) вектор A_p полностью сформирован.

6.3 Параллельная обработка разреженной структуры

Основная вычислительная фаза умножения симметричной разреженной матрицы на вектор реализована следующим образом:

```
#pragma omp for
for (int i = 0; i < n; i++) {
    double sum = 0.0;
    for (int k = A.rowPtr[i]; k < A.rowPtr[i+1]; k++) {
        int j = A.colIndex[k];
        double v = A.val[k];
```



```

        sum += v * p[j]; // Вклад в текущую строку i

        if (i != j) {
            local_Ap[tid][j] += v * p[i]; % Симметричный вклад в строку j
        }
    }
    local_Ap[tid][i] += sum;
}

```

Распределение итераций внешнего цикла по строкам i выполняется автоматически между доступными потоками. Каждый поток накапливает результаты в свою изолированную строку `local_Ap[tid]`, что исключает состояние гонки при обработке внедиагональных элементов.

7 Описание тестовых задач

Для всестороннего анализа корректности и эффективности разработанного параллельного алгоритма было проведено комплексное тестирование, разделенное на два этапа.

7.1 Тесты на проверку корректности

Целью данного этапа является верификация математической точности реализованного алгоритма и проверка соблюдения критерия сходимости. Для проведения экспериментов генерировались случайные разреженные квадратные матрицы со строгим диагональным преобладанием, что гарантирует их симметричность и положительную определенность.

Тестирование проводилось на наборах матриц различных размерностей:

- **Малые разреженные матрицы ($n \leq 1000$):** Использовались для базовой проверки логики итерационного процесса, корректности индексации массивов CRS (`rowPtr`, `colIndex`) и обработки граничных случаев (например, строк, содержащих только диагональный элемент).
- **Средние разреженные матрицы ($n = 10000$):** Применялись для подтверждения устойчивости алгоритма при большом числе итераций и проверки корректности работы критерия остановки `eps`.

Ответ считался корректным, если относительная норма остатка удовлетворяла условию:

$$\frac{\|Ax - b\|_2}{\|A\|_2} < 10^{-8} \quad (6)$$

7.2 Тесты производительности и масштабируемости

Для исследования скоростных характеристик

параллельной версии требовалось создать существенную вычислительную нагрузку, минимизирующую долю системных накладных расходов OpenMP на создание и синхронизацию потоков.

В соответствии с техническими требованиями, ограничения на размер задачи составляют $n \leq 100000$ при количестве ненулевых элементов $nz \leq 10^7$. Для профилирования масштабируемости была выбрана тестовая конфигурация со следующими параметрами:

- Размерность матрицы: $n = 100000$.
- Число ненулевых элементов: $nz = 10^7$.
- Диапазон изменения значений векторов: $|x_i| \leq 1, i = \overline{1, n}$.

Замеры общего времени выполнения алгоритма производились для последовательного режима (1 поток), а также для параллельного выполнения на конфигурациях с числом потоков $p \in \{2, 3, 4, 5, 6\}$. На основе полученных временных данных рассчитывались коэффициенты ускорения S_p и эффективности E_p , что позволяет оценить степень масштабируемости алгоритма и выявить точку насыщения пропускной способности памяти вычислительной системы.

8 Результаты экспериментов

8.1 Проверка корректности

Для верификации работы алгоритма проводилось вычисление относительной невязки полученного решения. В таблице 1 представлены результаты запуска на сгенерированных случайных матрицах с диагональным преобладанием.

Размер матрицы (n)	Итераций	Относительная невязка
10	11	8.524502×10^{-17}
50	22	1.342407×10^{-9}
100	24	$1.235791 \cdot 10^{-9}$
500	27	$5.558743 \cdot 10^{-9}$
1000	29	$9.014339 \cdot 10^{-9}$

Таблица 1: Результаты проверки корректности алгоритма

Как видно из таблицы, на всех тестовых размерностях алгоритм сходится, а относительная невязка строго удовлетворяет требуемому условию $\frac{\|Ax-b\|_2}{\|A\|_2} < 10^{-8}$. Небольшой рост невязки с увеличением размерности является математически ожидаемым и объясняется стандартным накоплением вычислительных погрешностей округления.

8.2 Тесты производительности и масштабируемости

Для исследования масштабируемости параллельной версии использовалась матрица размерности $n = [N]$, что позволило создать достаточную вычислительную нагрузку и нивелировать влияние накладных расходов OpenMP.

В таблице 2 представлены результаты замеров времени выполнения, а также рассчитанные значения ускорения ($S = T_1/T_p$) и эффективности ($E = S/p$).

Число потоков (p)	Время, с	Ускорение (S)	Эффективность (E), %
1	1.906	1.00	100.0
2	1.277	1.49	74.63
3	0.958	1.99	66.32
4	0.787	2.42	60.55
5	0.6810	2.8	55.98
6	0.621	3.07	51.15

Таблица 2: Производительность при $n = 100000$

Анализ полученных данных показывает уверенную масштабируемость алгоритма на выбранной архитектуре. Эффективность распараллеливания остается на высоком уровне и на всех тестовых конфигурациях строго превышает заданный техническим заданием порог в 50%. Использование локальных буферов позволило избежать блокировок и обеспечить равномерную загрузку вычислительных ядер.

9 Заключение

В рамках лабораторной работы была успешно реализована, протестирована и проанализирована параллельная версия метода сопряженных градиентов для систем линейных алгебраических уравнений с разреженными матрицами с использованием фреймворка OpenMP.

Применение симметричного формата CRS позволило существенно оптимизировать потребление оперативной памяти. Ключевая проблема распараллеливания — состояние гонки данных при записи симметричных вкладов — была эффективно решена за счет выделения частных потоковых массивов памяти. Это избавило код от необходимости использования ресурсоемких атомарных операций в основном вычислительном цикле.

По итогам тестирования подтверждено:

- **Корректность:** алгоритм стабильно сходится к правильному решению, выполняя жесткие критерии по относительной норме невязки ($< 10^{-8}$) на всех тестовых размерностях.
- **Производительность:** параллельная версия алгоритма демонстрирует высокую масштабируемость. Требование к сохранению эффективности распараллеливания на уровне не менее 50% выполнено в полном объеме для всех протестированных конфигураций потоков.

Поставленная задача решена полностью, заявленные ограничения по памяти и времени соблюдены.

10 Список литературы

1. Баркалов К.А. Параллельные численные методы: презентации лекций. ННГУ им. Н.И. Лобачевского.
2. Самарский А.А., Гулин А.В. Численные методы. М.: Наука, 1989.
3. Антонов А.С. Технологии параллельного программирования MPI и OpenMP: Учебное пособие. — М.: Издательство

A Приложение. Код

```
#include <iostream>
#include <vector>
#include <cmath>
#include <omp.h>
#include <random>
#include <iomanip>
#include <chrono>

struct CRSMatrix {
    int n;
    int m;
    int nz;
    std::vector<double> val;
    std::vector<int> colIndex;
    std::vector<int> rowPtr;
};

void SLE_Solver_CRS(CRSMatrix & A, double * b, double eps, int max_iter,
    double * x, int & count)
{
    int n = A.n;
    count = 0;

    std::vector<double> r(n, 0.0);
    std::vector<double> p(n, 0.0);
    std::vector<double> Ap(n, 0.0);

    int max_threads = omp_get_max_threads();
    std::vector<std::vector<double>> local_Ap(max_threads, std::vector<
double>(n, 0.0));

    double norm_b_sq = 0.0;

    #pragma omp parallel for reduction(+:norm_b_sq)
    for (int i = 0; i < n; i++) {
        x[i] = 0.0;
        r[i] = b[i];
        p[i] = b[i];
        norm_b_sq += b[i] * b[i];
    }

    double norm_b = sqrt(norm_b_sq);

    if (norm_b == 0.0) {
        return;
    }
}
```

```

}

double rr = norm_b_sq;

for (int iter = 0; iter < max_iter; iter++) {

#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int n_threads = omp_get_num_threads();

    for (int i = 0; i < n; i++) {
        local_Ap[tid][i] = 0.0;
    }

#pragma omp for
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int k = A.rowPtr[i]; k < A.rowPtr[i+1]; k++) {
            int j = A.colIndex[k];
            double v = A.val[k];

            sum += v * p[j];

            if (i != j) {
                local_Ap[tid][j] += v * p[i];
            }
        }
        local_Ap[tid][i] += sum;
    }

#pragma omp for
    for (int i = 0; i < n; i++) {
        double total = 0.0;
        for (int t = 0; t < n_threads; t++) {
            total += local_Ap[t][i];
        }
        Ap[i] = total;
    }
}

double pAp = 0.0;
#pragma omp parallel for reduction(+:pAp)
for (int i = 0; i < n; i++) {
    pAp += p[i] * Ap[i];
}

double alpha = rr / pAp;

```

```

    double dx_norm_sq = 0.0;
    #pragma omp parallel for reduction(+:dx_norm_sq)
    for (int i = 0; i < n; i++) {
        double dx = alpha * p[i];
        x[i] += dx;
        dx_norm_sq += dx * dx;
    }

    count++;

    double dx_norm = sqrt(dx_norm_sq);
    if (dx_norm / norm_b < eps) {
        break;
    }

    double rr_new = 0.0;
    #pragma omp parallel for reduction(+:rr_new)
    for (int i = 0; i < n; i++) {
        r[i] -= alpha * Ap[i];
        rr_new += r[i] * r[i];
    }

    double beta = rr_new / rr;

    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        p[i] = r[i] + beta * p[i];
    }

    rr = rr_new;
}

}

void multiply_CRS_vector(const CRSMatrix& A, const std::vector<double>& x
, std::vector<double>& res) {
    int n = A.n;
    fill(res.begin(), res.end(), 0.0);
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int k = A.rowPtr[i]; k < A.rowPtr[i+1]; k++) {
            int j = A.colIndex[k];
            double v = A.val[k];
            sum += v * x[j];
            if (i != j) {
                res[j] += v * x[i];
            }
        }
    }
}

```

```

        res[i] += sum;
    }
}

CRSMatrix generate_spd_crs(int n, int max_nz_per_row) {
    CRSMatrix A;
    A.n = n;
    A.m = n;
    A.rowPtr.push_back(0);
    A.nz = 0;

    std::mt19937 gen(144);
    std::uniform_real_distribution<double> dist_val(-10.0, 10.0);
    std::uniform_int_distribution<int> dist_col(0, n - 1);

    std::vector<double> diag_vals(n, 0.0);
    std::vector<std::vector<std::pair<int, double>>> rows(n);

    for (int i = 0; i < n; i++) {
        rows[i].push_back({i, 0.0});

        int nz_count = dist_col(gen) % max_nz_per_row;
        for (int k = 0; k < nz_count; k++) {
            int j = dist_col(gen);
            if (j > i) {
                double val = dist_val(gen);
                rows[i].push_back({j, val});
                diag_vals[i] += abs(val);
                diag_vals[j] += abs(val);
            }
        }
    }

    for (int i = 0; i < n; i++) {
        rows[i][0].second = diag_vals[i] + 10.0 + abs(dist_val(gen));

        for (auto& elem : rows[i]) {
            A.colIndex.push_back(elem.first);
            A.val.push_back(elem.second);
            A.nz++;
        }
        A.rowPtr.push_back(A.nz);
    }
    return A;
}

double calc_matrix_2norm(const CRSMatrix& A) {

```



```

int n = A.n;
std::vector<double> q(n, 1.0 / sqrt(n));
std::vector<double> z(n, 0.0);
double lambda = 0.0;

for (int iter = 0; iter < 100; iter++) {
    multiply_CRS_vector(A, q, z);
    double z_norm = 0.0;
    for (int i = 0; i < n; i++) z_norm += z[i] * z[i];
    z_norm = sqrt(z_norm);

    double new_lambda = 0.0;
    for (int i = 0; i < n; i++) {
        q[i] = z[i] / z_norm;
        new_lambda += q[i] * z[i]; // (Aq, q)
    }
    if (abs(new_lambda - lambda) < 1e-6) break;
    lambda = new_lambda;
}
return lambda;
}

double norm_2(const std::vector<double>& v) {
    double sum = 0.0;
    for (double val : v) sum += val * val;
    return sqrt(sum);
}

void test_correctness() {
    std::cout << "--- Test begin ---" << '\n';
    std::vector<int> test_sizes = {10, 50, 100, 500, 1000};

    for (int n : test_sizes) {
        CRSMatrix A = generate_spd_crs(n, 10);

        std::vector<double> x_true(n);
        std::mt19937 gen(123);
        std::uniform_real_distribution<double> dist_x(-1.0, 1.0);
        for (int i = 0; i < n; i++) x_true[i] = dist_x(gen);

        std::vector<double> b(n);
        multiply_CRS_vector(A, x_true, b);

        std::vector<double> x(n, 0.0);
        int iters = 0;

        SLE_Solver_CRS(A, b.data(), 1e-10, 10000, x.data(), iters);
    }
}

```

```

        std::vector<double> Ax(n);
        multiply_CRS_vector(A, x, Ax);

        std::vector<double> residual(n);
        for (int i = 0; i < n; i++) residual[i] = Ax[i] - b[i];

        double norm_res = norm_2(residual);
        double norm_A = calc_matrix_2norm(A);
        double rel_error = norm_res / norm_A;

        std::cout << "n = " <<std::setw(5) << n
                    << " | Num inters: " << std::setw(4) << iters
                    << " | Rel. error: " << std::scientific << rel_error;

        if (rel_error < 1e-8) std::cout << " [PASSED]" << '\n';
        else std::cout << " [FAILED]" << '\n';
    }
    std::cout << '\n';
}

void test_performance() {
    std::cout << "--- Perf test begin ---" << '\n';
    int n = 100000;
    int max_nz_per_row = 100;
    std::cout << " mat size " << n << "x" << n << "..." << std::flush;

    CRSMatrix A = generate_spd_crs(n, max_nz_per_row);
    std::cout << " A.nz: " << A.nz << '\n';

    std::vector<double> x_true(n, 1.0);
    std::vector<double> b(n);
    multiply_CRS_vector(A, x_true, b);
    std::vector<double> x(n, 0.0);

    std::vector<int> threads = {1, 2, 3, 4, 5, 6};
    double time_seq = 0.0;

    std::cout << std::fixed << std::setprecision(4);

    for (int p : threads) {
        omp_set_num_threads(p);
        fill(x.begin(), x.end(), 0.0);
        int iters = 0;
        auto start = std::chrono::steady_clock::now();
        SLE_Solver_CRS(A, b.data(), 1e-8, 1000, x.data(), iters);
        double time_spent = std::chrono::duration_cast<std::chrono::
milliseconds>(std::chrono::steady_clock::now() - start).count() /
1000.1;
    }
}

```

```

        if (p == 1) {
            time_seq = time_spent;
        }
        double speedup = time_seq / time_spent;
        double efficiency = (speedup / p) * 100.0;

        std::cout << std::setw(10) << p << std::setw(15) << time_spent
            << std::setw(15) << speedup << std::setw(19) << efficiency <<
            "%" << '\n';

    }
}

int main() {
    test_correctness();
    test_performance();
    return 0;
}

```